

REQUIREMENT ANALYSIS

GROUP 10

OmniTrade Hub

1. Project Lifecycle Position

Requirement analysis is the foundational first step of the OmniTrade Hub development lifecycle. It precedes technical architecture decisions, database entity modeling, and sprint planning, ensuring we fully understand the business problem before writing code.

Business Problem → Requirement Analysis → System Design → Database Design → Agile Sprint Planning → Development

2. Business Problem and Goal

Independent merchants struggle to find a centralized, standardized digital storefront to sell products, while buyers must navigate fragmented channels. Furthermore, platform operators face limited visibility into multi-vendor order routing, fulfillment tracking, and inventory synchronization, leading to stock discrepancies and delayed fulfillment.

Our goal is to build a centralized **Multi-Vendor Digital Marketplace & Order Fulfillment System (OmniTrade Hub)** that enables independent merchants to manage product catalogs, allows customers to purchase items across multiple vendors in a single checkout session, and provides administrators with comprehensive oversight of orders, vendors, and fulfillment workflows.

3. Stakeholders

<i>Stakeholder</i>	<i>Interest</i>
Buyers / Customers	Browse product catalogs across multiple vendors, manage a unified shopping cart, place secure orders, and track fulfillment status.
Vendors / Merchants	Register their storefronts, manage independent product listings, track inventory stock levels, and process assigned order items.
Platform Administrators	Manage platform users, oversee vendor approvals, monitor global Gross Merchandise Value (GMV), and handle system configurations.
Warehouse / Fulfillment Staff	Track order fulfillment pipelines, update shipping statuses, and manage stock deductions or return rollbacks.

4. Primary Actors

- **Customer** (End-user buying products)
- **Vendor** (Merchant managing products and vendor-specific orders)
- **Administrator** (Platform operator managing users and global settings) (Note: SQL Server and external payment/notification gateways form part of the infrastructure and external systems, not direct user actors.)

5. Functional Requirements

Actor	Functional Requirements
Customer	Register/login; browse and filter products; add items from multiple vendors to a unified cart; place orders; view order history and fulfillment statuses.
Vendor	Manage storefront profile; create, update, and delete product listings; view incoming orders containing their items; update item fulfillment status.
Administrator	Manage user roles and permissions; approve or suspend vendor accounts; view platform-wide analytics dashboards (GMV, total orders, active vendors).

6. Non-Functional Requirements

Response Time: Catalog search and dashboard queries should return results within 2 seconds under standard load.

Security: Role-based access control (RBAC) ensuring vendors can only modify their own products/orders, and customers can only access their personal carts and purchase history.

Reliability & Data Integrity: Strict database transaction handling (ACID compliance via EF Core) to ensure inventory stock is correctly deducted upon order placement and rolled back safely if an order is cancelled.

Scalability: Decoupled three-tier architecture (Web API backend + MVC client) allowing independent scaling of the backend services and frontend presentation layer.

7. Sample User Stories

Customer: *As a customer, I want to add products from multiple vendors into a single shopping cart so that I can checkout seamlessly in one transaction.*

Vendor: *As a vendor, I want to update my product inventory levels so that customers do not purchase out-of-stock items.*

Administrator: *As an administrator, I want to monitor platform-wide order metrics so that I can track overall marketplace performance.*

8. Acceptance Criteria

User Story: *As a customer, I want to place an order from my shopping cart so that I can purchase desired products.*

#	Acceptance Criterion
1	<i>The customer must be authenticated and logged into the system.</i>
2	<i>The shopping cart must contain at least one valid product item with available stock.</i>
3	<i>Submitting the checkout form must generate a unique order reference number.</i>
4	<i>The initial order status must automatically set to Pending.</i>
5	<i>Product stock quantities must automatically decrement based on ordered quantities.</i>
6	<i>The customer should be able to view the newly created order in their order history dashboard.</i>

9. Given-When-Then Structure

Given *the customer is logged in with active items in their shopping cart.*

When *the customer submits the checkout form.*

Then *the system should create the order, generate a unique order ID, and deduct the corresponding stock.*

And *set the initial order status to **Pending**.*